

Aula 05: Tipos de dados em Python

Características, conversão e operações

Maurício da Silva Pires
mspires@id.uff.br

TCC00326 - Programação de computadores
Niterói, Rio de Janeiro

Revisitando as aulas anteriores

- ⌋ Nas aulas anteriores discutimos uma série de conceitos básicos sobre Python
 - * Características da linguagem
 - * Estrutura de um programa Python
 - * Características dos processos de compilação e interpretação
- ⌋ Trabalhamos com um modelo de computação em que um programa de computador é uma implementação direta de um algoritmo
- ⌋ Nesta aula, vamos focar nos mais básicos tipos de dados do Python

Características do Python

- ☾ Python é uma linguagem de programação multiparadigma, ou seja, segue diferentes perspectivas para a construção de softwares
 - * Um destes paradigmas é chamado **Orientação a objetos**
 - * Neste paradigma, todos os dados pertencem a alguma **classe**
 - * **Classe** é o equivalente a um tipo de dados, definindo faixas de valores possíveis e comportamentos que estes dados podem performar
 - * Os valores que manipulamos, por sua vez, são **objetos**, ou instâncias de alguma classe
- ☾ A classe define uma família de objetos que possuem características e comportamentos semelhantes

Características do Python

- ⌋ Ao longo deste curso, vamos focar em dois grupos específicos de dados
 - * Na primeira metade, vamos aprender um pouco sobre números inteiros, números reais, valores-verdade, básico de textos e intervalos inteiros
 - * Na segunda metade, exploraremos manipulação avançada de textos, listas, conjuntos, tuplas e dicionários
- ⌋ O conteúdo é cumulativo e, por isso, começamos com as classes mais “amigáveis” de objetos
- ⌋ Os números inteiros, números reais, valores-verdade e textos são objetos imutáveis, ou seja, não é possível alterar nenhuma parte deles

Números inteiros

- Objetos da classe **int** denotam números inteiros
 - Os literais de números inteiros se escrevem da forma como usualmente escrevemos na matemática
 - O literal de um inteiro é uma sequência de algarismos de 0 a 9 precedida ou não do sinal —
- Podemos escrever um número inteiro em outras bases adicionando os prefixos 0b (binário), 0o (octal) e 0x (hexadecimal)
- A precisão dos inteiros é arbitrária e limitada à memória do computador a partir do Python 3.0

Números inteiros

- ⌋ Números escritos em base 2, 8 ou 16 são automaticamente convertidos para a base 10 durante a atribuição
 - * O interpretador decodifica e atribui à variável um objeto inteiro de base 10

```
x = 0b100101
print(x)                # 37
```

```
y = 0o234
print(y)                # 156
```

```
z = 0xABC
print(z)                # 2748
```

- Python oferece as funções **bin**, **oct** e **hex** para transformar um inteiro em sua representação textual nas bases 2, 8 e 16, respectivamente

✱ O retorno dessas funções é sempre texto

```
x = bin(37)
print(x)                # 0b100101
```

```
y = oct(156)
print(y)                # 0o234
```

```
z = hex(2748)
print(z)                # 0xabc
```

- Objetos da classe **float** denotam números reais
 - Os literais de números reais se escrevem da forma como usualmente escrevemos na matemática, só que usando um ponto para separar a parte inteira da fracionária
 - Se não colocarmos uma parte inteira no número, o Python interpreta que esta parte é igual a zero
- Pode apresentar alguns problemas de precisão devido à forma como os números reais são representados internamente
 - Por exemplo, a soma $0.2 + 0.1$ em vez de resultar em 0.3 resulta no número 0.30000000000000004
 - Este é um tipo de dado sujeito a *overflow* na representação
 - Em caso de overflow, a operação retorna **inf** (infinito)

- Para saber os limites máximo e mínimo da representação de ponto flutuante, execute o seguinte

```
import sys
print(sys.float_info.max)    # Limite máximo
print(sys.float_info.min)    # Limite mínimo
```

- Em geral, os erros de precisão dos números reais podem ser corrigidos de duas formas
 - * **round(*n*,*x*)** arredonda um número real *n* para *x* casas decimais. Se *x* for omitido, o número real é arredondado para um inteiro
 - * Formatando o texto na hora de exibir (o que vamos deixar para explorar com calma na Aula 09)

Operadores aritméticos

- ⌋ Em geral, as operações aritméticas binárias podem ser aplicadas a dados numéricos de tipos diferentes
 - * Se ambos os operadores forem inteiros, o resultado também é um inteiro. Se ao menos um dos operandos for real, então o resultado também será real
 - * Isso se aplica a soma (+), subtração (−) e multiplicação (*)
 - * Divisão inteira (//) segue a regra geral
 - * Divisão real (/) sempre retorna um real independente do tipo dos argumentos
 - * O operador % retorna o resto da divisão inteira e só admite operandos inteiros
 - * O operador ** indica a potenciação, onde o operando antes é a base e o operando depois é o expoente

Operadores de atualização aritmética

- ﴿ Podemos combinar os operadores aritméticos com o operador de atribuição para simplificar a escrita

* Seja $\star \in \{+, -, *, /, //, \%, **\}$, *var* uma variável e *n* um número inteiro/real. As seguintes notações são equivalentes

$$\text{var } \star = n \quad \equiv \quad \text{var} = \text{var } \star n$$

...

x, y, z = 24, 3, 14

x %= 2

y **= 6

z /= 4

x = x % 2 -> 0

y = y ** 6 -> 729

z = z / 4 -> 3.5

...

- Objetos da classe **bool** denotam valores-verdade **True** e **False**
 - * Todo e qualquer objeto em Python pode ser usado em testes de valor-verdade, desde que sua classe implemente um método **bool()** ou um método **len()**
- Objetos que são considerados equivalentes ao **False**
 - * A constante **None**
 - * *String* vazia, lista vazia, conjunto vazio, dicionário vazio
 - * O número zero seja na versão inteira ou na versão real
- As condições que avaliamos nos desvios condicionais e na repetição indefinida 0+, na verdade, depende do valor **True** ou **False**

Operadores lógicos

- ☾ Só existem 3 operadores lógicos: a negação **not**, a conjunção **and** (e-lógico) e a disjunção **or** (ou-lógico)

A	B	not A	A and B	A or B
True	True	False	True	True
True	False	False	False	True
False	True	True	False	True
False	False	True	False	False

- * A negação inverte o valor-verdade
- * A conjunção é verdadeira se ambos forem verdade
- * A disjunção é verdadeira se ao menos um é verdade

⌋ Operações relacionais são operações de comparação

- * Podemos comparar números com números e textos com textos (ordenação lexicográfica)
- * Sim, é possível comparar números inteiros com números reais
- * O resultado é sempre **True** ou **False**

Operador	Significado	Operador	Significado
==	Igualdade	!=	Desigualdade
<	Menor que	<=	Menor ou igual a
>	Maior que	>=	Maior ou igual a

- Objetos da classe **str** denotam textos
 - Os literais do tipo texto sempre são colocados entre aspas duplas ou aspas simples
- Os caracteres de uma *string* (texto) são numerados, da esquerda para direita de 0 a $n - 1$, onde n é o total de letras
 - Índices negativos de -1 a $-n$ fazem acesso da direita para a esquerda
 - $p[i]$ indica o caracter de índice i da *string* p
- A função **len(p)** retorna o número de caracteres da *string* p

Textos formatados

- Podemos adicionar algumas formatações no texto usando o método **.format(args)**
 - * **args** é a sequência de argumentos usados para a formatação
 - * Os locais onde uma informação deve ser inserida é marcada com {}
 - * O preenchimento em questão pode ser feito usando notação posicional ou indicando o índice do argumentos (começando em 0)

```
x, y = 2, 14
```

```
print("{} é múltiplo de {}".format(y,x))  
# 14 é múltiplo de 2
```

```
print("{1} é múltiplo de {0}".format(x,y))  
# 14 é múltiplo de 2
```


Operação de conversão de dados

- Podemos modificar o tipo de algumas variáveis a partir de operações de conversão
 - Esse tipo de operação, conhecida como casting, em Python, é feita a partir dos métodos construtores de cada classe
- Um **método construtor** é uma operação pré-definida para uma determinada classe (tipo) que cria um objeto
 - O método construtor usa o nome da própria classe
 - `int( )`, `bool( )`, `str( )`, `float( )`
- Lembre-se que o método **input** sempre retorna dados do tipo texto
 - Podemos ler números a partir da operação de conversão

Exemplo

☾ Leia do usuário dois números a e b e decida se a soma deles é um número par ou ímpar

✱ Observe que cada número é lido em uma linha

✱ Os dados lidos foram convertidos para o tipo inteiro

```
if __name__ == "__main__":  
    a = int(input())  
    b = int(input())  
    if (a + b)%2 == 0 :  
        print("A soma é par")  
    else :  
        print("A soma é ímpar")
```

Intervalos de inteiros

- ⌋ A função **range** gera um tipo de dado chamado **intervalo de inteiros**

✱ Este intervalo é a listagem dos inteiros entre dois valores

```
range(x)           # de 0 a x, excluindo x, de 1 em 1
range(x, y)        # de x a y, excluindo y, de 1 em 1
range(x, y, p)     # de x a y, excluindo y, de p em p
```

- ⌋ Podemos usar essa função para gerar, por exemplo, progressões aritméticas
- ⌋ Para usar a repetição iteradora fazendo uma contagem, passamos como coleção iterável um intervalo de inteiros

Exemplo

☞ Vamos escrever duas repetições definidas

- * Uma que executa 10 vezes e, a cada passagem da repetição, o imprime uma linha e o número da linha
- * Note que $\text{range}(10) = 0, 1, 2, 3, 4, 5, 6, 7, 8, 9$
- * Lembre-se que, na repetição iteradora, a variável vai assumir cada um dos valores na coleção iterável que usamos

```
if __name__ == "__main__":  
    for i in range(10):  
        print("Linha número {}".format(i))  
  
    palavra = "solúvel"  
    # vai imprimir uma letra por linha  
    for letra in palavra:  
        print(letra)
```

Testando tipo de uma variável

- ⌋ O tipo de uma variável depende do valor atrelado a ela em um dado momento, já que tipagem no Python é dinâmica
- ⌋ Para testar o tipo de uma variável em um dado momento, usamos o método **isinstance** que recebe dois argumentos
 - * O método recebe dois argumentos: uma variável e uma classe ou tupla de classes
 - * Ele retorna **True** se a variável é de algum dos tipos descritos

```
if __name__ == "__main__":  
    x = 10  
    isinstance(x, int)           # True  
    isinstance(x, bool)         # False  
    x = str(x)  
    isinstance(x, (str, int))    # True
```

Testando tipo de uma variável

☾ Outra alternativa para o teste de variável é o método **type**

✱ O método retorna uma classe

✱ Para comparar classes, usamos o operador **is**

```
if __name__ == "__main__":  
    x = 10  
    if type(x) is int :  
        print("A variável é inteira")  
    else :  
        print("A variável não é inteira")  
    x = float(x)  
    if type(x) is float :  
        print("Agora ela é real")
```

Precedência de operadores

- ⌈ Expressões podem envolver uma série de operadores de categorias diferentes (aritméticos, relacionais e lógicos)
- ⌈ Define-se uma precedência de operadores para o caso de expressões que não utilizam parênteses para definir a ordem da execução
 - * Parênteses possuem a maior prioridade
 - * Operadores aritméticos
 - * Operadores relacionais
 - * Operadores lógicos
- ⌈ Em alguns casos, há regras específicas para operados dentro de uma mesma categoria

Precedência de operadores

⌞ Precedência dos operadores aritméticos

1. Potenciação
2. Multiplicação, divisões e módulo
3. Adição e subtração

⌞ Os operadores relacionais possuem mesma precedência

⌞ Precedência dos operadores lógicos

1. Negação (not)
2. Conjunção (and)
3. Disjunção (or)

Precedência de operadores

- Se uma expressão é escrita sem o uso dos parênteses, ela é avaliada segundo a precedência dos operadores
- No caso de a expressão envolver operadores que possuem a mesma precedência, o valor da expressão é determinado a partir da execução das operações da esquerda para a direita, na ordem em que aparecem
- Exemplo:

$x, y, z = 10, -2, 5$

$w = x * y < z / x \text{ or } x / y > z * x \text{ and } z * y < x$

Precedência de operadores

$w = x * y < z / x \text{ or } x / y > z * x \text{ and } z * y < x$

Precedência de operadores

$$\begin{aligned}w &= x * y < z / x \text{ or } x / y > z * x \text{ and } z * y < x \\&= 10 * (-2) < 5/10 \text{ or } 10/(-2) > 5 * 10 \text{ and } 5 * (-2) < 10\end{aligned}$$

Precedência de operadores

$$\begin{aligned}w &= x * y < z / x \text{ or } x / y > z * x \text{ and } z * y < x \\&= 10 * (-2) < 5/10 \text{ or } 10/(-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 5/10 \text{ or } 10/(-2) > 5 * 10 \text{ and } 5 * (-2) < 10\end{aligned}$$

Precedência de operadores

$$\begin{aligned}w &= x * y < z / x \text{ or } x / y > z * x \text{ and } z * y < x \\&= 10 * (-2) < 5/10 \text{ or } 10/(-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 5/10 \text{ or } 10/(-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } 10/(-2) > 5 * 10 \text{ and } 5 * (-2) < 10\end{aligned}$$

Precedência de operadores

$$\begin{aligned}w &= x * y < z / x \text{ or } x / y > z * x \text{ and } z * y < x \\&= 10 * (-2) < 5 / 10 \text{ or } 10 / (-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 5 / 10 \text{ or } 10 / (-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } 10 / (-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 5 * 10 \text{ and } 5 * (-2) < 10\end{aligned}$$

Precedência de operadores

$$\begin{aligned}w &= x * y < z / x \text{ or } x / y > z * x \text{ and } z * y < x \\&= 10 * (-2) < 5 / 10 \text{ or } 10 / (-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 5 / 10 \text{ or } 10 / (-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } 10 / (-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 50 \text{ and } 5 * (-2) < 10\end{aligned}$$

Precedência de operadores

$$\begin{aligned}w &= x * y < z / x \text{ or } x / y > z * x \text{ and } z * y < x \\&= 10 * (-2) < 5/10 \text{ or } 10/(-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 5/10 \text{ or } 10/(-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } 10/(-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 50 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 50 \text{ and } (-10) < 10\end{aligned}$$

Precedência de operadores

$$\begin{aligned}w &= x * y < z / x \text{ or } x / y > z * x \text{ and } z * y < x \\&= 10 * (-2) < 5 / 10 \text{ or } 10 / (-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 5 / 10 \text{ or } 10 / (-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } 10 / (-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 50 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 50 \text{ and } (-10) < 10 \\&= \text{True or } (-5) > 50 \text{ and } (-10) < 10\end{aligned}$$

Precedência de operadores

$$\begin{aligned}w &= x * y < z / x \text{ or } x / y > z * x \text{ and } z * y < x \\&= 10 * (-2) < 5/10 \text{ or } 10/(-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 5/10 \text{ or } 10/(-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } 10/(-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 50 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 50 \text{ and } (-10) < 10 \\&= \text{True or } (-5) > 50 \text{ and } (-10) < 10 \\&= \text{True or False and } (-10) < 10\end{aligned}$$

Precedência de operadores

$$\begin{aligned}w &= x * y < z / x \text{ or } x / y > z * x \text{ and } z * y < x \\&= 10 * (-2) < 5 / 10 \text{ or } 10 / (-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 5 / 10 \text{ or } 10 / (-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } 10 / (-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 50 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 50 \text{ and } (-10) < 10 \\&= \text{True or } (-5) > 50 \text{ and } (-10) < 10 \\&= \text{True or False and } (-10) < 10 \\&= \text{True or False and True}\end{aligned}$$

Precedência de operadores

$$\begin{aligned}w &= x * y < z / x \text{ or } x / y > z * x \text{ and } z * y < x \\&= 10 * (-2) < 5 / 10 \text{ or } 10 / (-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 5 / 10 \text{ or } 10 / (-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } 10 / (-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 50 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 50 \text{ and } (-10) < 10 \\&= \text{True or } (-5) > 50 \text{ and } (-10) < 10 \\&= \text{True or False and } (-10) < 10 \\&= \text{True or False and True} \\&= \text{True or False}\end{aligned}$$

Precedência de operadores

$$\begin{aligned}w &= x * y < z / x \text{ or } x / y > z * x \text{ and } z * y < x \\&= 10 * (-2) < 5 / 10 \text{ or } 10 / (-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 5 / 10 \text{ or } 10 / (-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } 10 / (-2) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 5 * 10 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 50 \text{ and } 5 * (-2) < 10 \\&= (-20) < 0.5 \text{ or } (-5) > 50 \text{ and } (-10) < 10 \\&= \text{True or } (-5) > 50 \text{ and } (-10) < 10 \\&= \text{True or False and } (-10) < 10 \\&= \text{True or False and True} \\&= \text{True or False} \\&= \text{True}\end{aligned}$$

